

CineDetetive — Documentação do Projeto

Oct 1, 2026 · @Victor

Visão geral

O CineDetetive é um chat que descobre o nome de um filme a partir de uma descrição vaga, perguntando mais detalhes quando não tem certeza. O usuário escreve algo como "um filme dos anos 90 em que o cara vive o mesmo dia várias vezes" e recebe o filme, com pôster, ano e elenco.

O projeto existe para praticar, num sistema completo, as cinco frentes de Engenharia de IA:

- **RAG:** busca semântica e híbrida em cerca de 35 mil sinopses de filmes, com modelos rodando localmente.
- **Tool calling:** o agente consulta a API do TMDb para filtrar e confirmar candidatos.
- **LangGraph:** fluxo com decisões, laços e perguntas de esclarecimento.
- **Evals:** acerto medido de forma objetiva (o filme certo apareceu ou não).
- **Observabilidade:** cada resposta rastreada no Langfuse, com tokens e latência.

Princípios de simplicidade

- **Sem login:** qualquer pessoa abre o site e usa.
- **Histórico só no navegador:** as conversas ficam guardadas na máquina do usuário pelo maior tempo que o navegador permitir. Guia anônima ou outro navegador começa uma sessão nova, vazia.
- **Servidor sem estado de conversa:** o servidor não guarda histórico; a cada mensagem o navegador envia o que o agente precisa.
- **IA local:** o LLM é um Llama rodando na própria máquina via Ollama, assim como os embeddings e o reranker. Nenhum texto do usuário vai para um provedor de LLM externo.

Escopo

Fase	Entrega
MVP	Identificar o filme, perguntar quando houver dúvida, aceitar "não é esse", histórico no navegador, feedback anônimo, evals e observabilidade

Fase	Entrega
Fase 2	Onde assistir no Brasil, filmes parecidos, cobertura de filmes recentes, exportar e importar conversas, deploy

Fora do escopo: contas de usuário, sincronizar histórico entre dispositivos, app mobile e reconhecimento de filme por imagem ou áudio.

Como funciona

O usuário descreve o que lembra, o agente busca candidatos nas sinopses, confirma no TMDb e responde só quando tem confiança; se não tiver, faz uma pergunta.

1. O usuário abre o site. Na primeira visita, o navegador gera um `session_id` aleatório e o guarda; não há cadastro.
2. Descreve o filme em português, do jeito que lembrar.
3. O front envia ao agente a mensagem, o histórico recente e o estado da busca (filtros, filmes recusados, perguntas já feitas).
4. O agente extrai pistas (década, gênero, ator, cenas) e traduz a busca para inglês, idioma das sinopses.
5. Busca os trechos de sinopse mais parecidos, reordena com o reranker e agrupa por filme.
6. Usa as tools do TMDb para filtrar por ano, ator ou gênero e buscar pôster e elenco.
7. Avalia a confiança no melhor candidato:
 - alta: responde com o filme;
 - média: mostra os 3 mais prováveis;
 - baixa: faz uma pergunta de esclarecimento, no máximo 3 vezes.
8. O agente devolve a resposta e o estado atualizado; o front guarda os dois no navegador.
9. O usuário confirma, diz "não é esse" (o filme é excluído e a busca continua) ou dá nota à resposta, que é gravada de forma anônima.

Conversa de exemplo

Turno	Quem	O que acontece
1	Usuário	"Um filme em que o cara acorda todo dia no mesmo dia, acho que tinha uma marmota"

Turno	Quem	O que acontece
1	Agente	Busca "man relives the same day, groundhog"; confiança alta
1	Agente	Responde: <i>Feitiço do Tempo</i> (Groundhog Day, 1993), com pôster e elenco
2	Usuário	"Não, era mais recente, tinha alienígenas"
2	Agente	Exclui o filme, adiciona "aliens" e ano $\geq$ 2000, busca de novo
2	Agente	Confiança média: mostra <i>No Limite do Amanhã</i> (Edge of Tomorrow, 2014) e mais 2 opções

Stack e linguagens

São duas linguagens: TypeScript no front e Python no agente. Sem login nem histórico no servidor, não há mais uma API de produto separada; o front fala direto com o serviço do agente.

Camada	Tecnologia	Linguagem	Por quê
Frontend (chat)	Next.js, React, Tailwind	TypeScript	Streaming da resposta palavra por palavra, UI de chat
Histórico local	IndexedDB via Dexie.js	TypeScript	Guarda conversas no navegador, com mais espaço e durabilidade que o localStorage
Serviço do agente	FastAPI, LangGraph, LangChain	Python 3.12	Onde está o ecossistema de IA; serve o agente por streaming (SSE)
Servidor de modelos	Ollama	—	Roda o Llama e os embeddings localmente, com API HTTP simples
LLM	Llama Instruct com suporte a tools (ex.: Llama 3.1 8B) via <code>langchain-ollama</code>	—	Local e gratuito; trocável pela abstração do LangChain

Camada	Tecnologia	Linguagem	Por quê
Embeddings	bge-m3 via Ollama	—	Multilíngue: usuário em português, sinopses em inglês
Reranker	bge-reranker-v2-m3 (sentence-transformers)	Python	Melhora a ordem dos candidatos, roda local
Banco de dados	PostgreSQL 16 + pgvector	SQL	Vetores, busca textual e feedback anônimo num banco só
Cache	Cache em memória com TTL (cachetools)	Python	Evita repetir chamadas ao TMDb, sem precisar de Redis
Observabilidade	Langfuse	—	Traces, latência, prompts versionados, datasets de avaliação
Evals	pytest + scripts próprios, DeepEval para LLM-as-judge	Python	Métricas objetivas de acerto e juiz para perguntas e respostas
Infra local	Docker Compose	YAML	Sobe tudo com um comando
CI	GitHub Actions	YAML	Roda testes e evals de recuperação a cada pull request

Projetos do repositório

O repositório é um monorepo com duas aplicações, um pipeline de ingestão, uma suíte de evals e a infraestrutura compartilhada.

Projeto	Pasta	Responsabilidade	Fala com
web — front do chat	apps/web	Lista de conversas, chat com streaming, cards de filme, botões "é esse" / "não é esse" e nota; guarda tudo no IndexedDB do navegador	agent

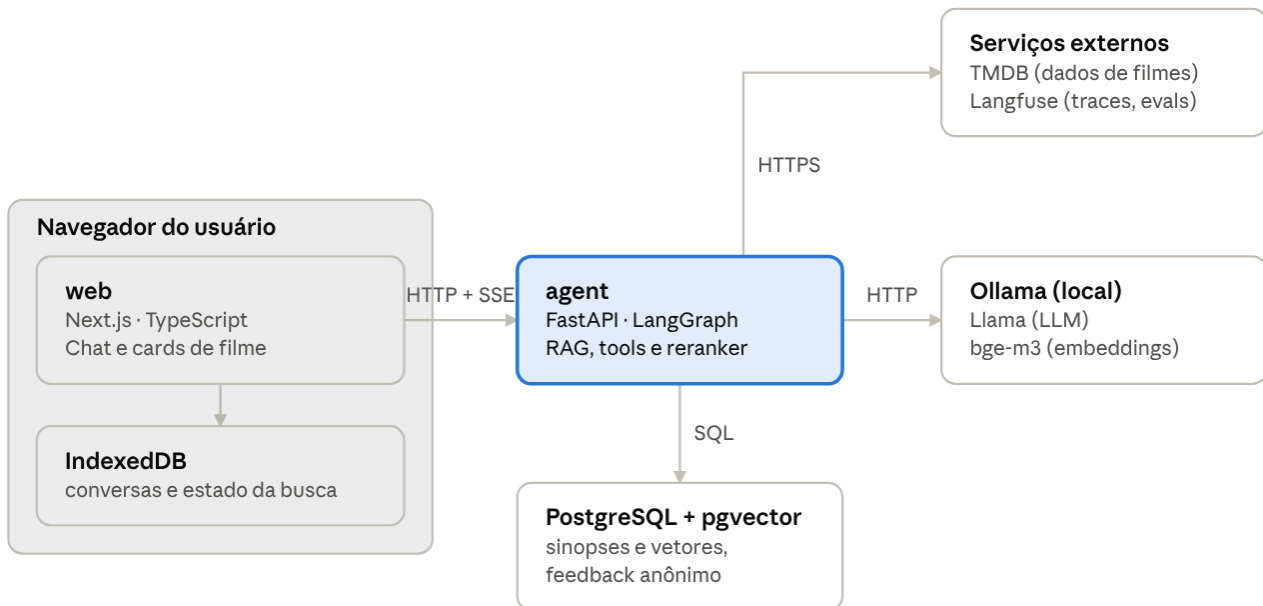
Projeto	Pasta	Responsabilidade	Fala com
agent — serviço de IA	<code>apps/agent</code>	Grafo do LangGraph, tools, RAG, streaming de eventos, recebimento do feedback anônimo	Ollama, Postgres, TMDB, Langfuse
ollama — modelos locais	serviço do Docker	Serve o Llama e o modelo de embeddings	agent, ingestion
ingestion — carga de dados	<code>apps/agent/ingestion</code>	Baixa as sinopses, limpa, liga cada filme ao TMDB, quebra em trechos e gera embeddings	Ollama, Postgres, TMDB
evals — avaliação	<code>apps/agent/evals</code>	Gera e guarda os casos de teste, roda as avaliações e publica os resultados	agent, Langfuse
infra — ambiente	<code>infra</code>	Docker Compose, script de criação do banco, variáveis de ambiente	todos

O agent não sabe quem é o usuário; ele só recebe um `session_id` aleatório gerado pelo navegador, usado para limitar requisições e agrupar traces no Langfuse.

Arquitetura

O navegador guarda as conversas e chama o agente; o agente usa modelos locais no Ollama e só sai da máquina para falar com o TMDB e o Langfuse.

Tudo roda na sua máquina, exceto TMDB e Langfuse



arquitetura · navegador, agente, Ollama, Postgres e 2 serviços externos

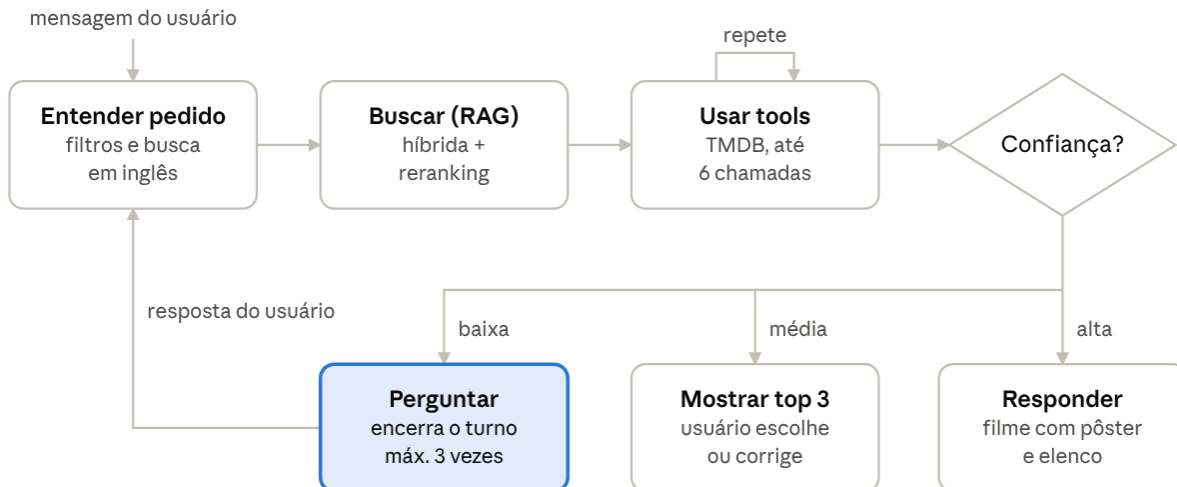
O servidor não guarda histórico. Cada requisição leva o que o agente precisa, e cada resposta devolve o estado atualizado para o navegador gravar no IndexedDB. Fechar uma guia anônima ou trocar de navegador apaga ou esconde esse histórico, e a próxima visita começa uma sessão nova.

Observação: com o Langfuse Cloud, os textos das conversas aparecem nos traces fora da máquina. Para manter tudo local, use o Langfuse hospedado por você (fase 2) ou desligue o envio de entradas e saídas nos traces.

Fluxo do agente no LangGraph

O grafo tem um caminho principal (entender, buscar, usar tools, avaliar) e três saídas conforme a confiança; a saída "baixa" encerra o turno com uma pergunta, e a resposta do usuário inicia o próximo turno do começo.

Na dúvida, o agente pergunta em vez de chutar



grafo do agente · 7 nós, 1 decisão, 1 laço de esclarecimento

O grafo roda sem checkpoint no servidor. O estado da busca (tabela abaixo, exceto `messages`) volta para o navegador no evento `final` e é reenviado na mensagem seguinte; o histórico enviado se limita às últimas 10 mensagens. Depois de 3 perguntas, a saída "baixa" vira "média" e o agente mostra o que tem.

Estado do grafo

Campo	Tipo	Para quê
<code>messages</code>	lista de mensagens	Histórico da conversa
<code>consulta_en</code>	texto	Busca reescrita em inglês
<code>filtros</code>	objeto (ano, gêneros, pessoas)	Restrições extraídas da fala
<code>candidatos</code>	lista de filmes com nota	Resultado da busca e das tools
<code>excluidos</code>	lista de <code>tmdb_id</code>	Filmes que o usuário recusou
<code>confianca</code>	alta, média ou baixa	Decide o próximo nó
<code>perguntas_feitas</code>	número	Limita as perguntas a 3
<code>intencao</code>	descrever, recusar, confirmar, fora do escopo	Definida no primeiro nó

Regras dos nós

- **Entender pedido** usa saída estruturada. Se a intenção for "recusar" ("não é esse"), adiciona o filme a `excluidos`; se for "fora do escopo", responde direto e encerra o turno.
- **Avaliar confiança** é um LLM como juiz que compara a descrição com as sinopses dos 3 primeiros e devolve nível e justificativa.
- **Perguntar** escolhe a pergunta que mais separa os candidatos (por exemplo, a década, quando eles são de épocas diferentes).
- **Responder** só usa dados vindos do TMDb ou do índice, nunca da memória do modelo.

Dados e fontes

As sinopses vêm de um dataset público da Wikipedia e são a base do RAG; o TMDb entra em tempo real para confirmar, filtrar e enriquecer.

Fonte	O que traz	Como entra	Cuidados
Wikipedia Movie Plots (Kaggle)	Cerca de 35 mil filmes: título, ano, origem, diretor, elenco, gênero e sinopse longa em inglês	Ingestão única, vira vetores no pgvector	Texto sob licença CC BY-SA: citar a Wikipedia; cobre bem até meados da década de 2010
TMDb API	Título em português, pôster, elenco, pessoas, busca por título	Tools chamadas pelo agente, com cache em memória	Chave gratuita para uso não comercial; exige exibir a atribuição do TMDb no app
TMDb <i>watch providers</i> (fase 2)	Onde assistir no Brasil	Tool <code>onde_assistir</code>	Dados fornecidos pelo JustWatch, que também exige atribuição

Ligação entre as fontes

Na ingestão, cada filme da Wikipedia é procurado no TMDb por título e ano. O `tmdb_id` encontrado vai para os metadados, junto com uma nota de confiança do pareamento. Filmes sem par seguro continuam no índice, mas aparecem sem pôster.

Antes de começar, confira os termos atuais do TMDb e a licença do dataset no Kaggle, pois podem ter mudado.

Pipeline de RAG

O RAG tem duas metades: a ingestão, que roda uma vez e prepara o índice, e a busca, que roda a cada pergunta e devolve filmes, não trechos.

Ingestão (uma vez)

1. **Baixar e limpar:** remover sinopses com menos de 300 caracteres, duplicatas e marcações da Wikipedia (referências como [1]).
2. **Ligar ao TMDb:** buscar `tmdb_id` por título e ano, guardar a confiança do pareamento.
3. **Quebrar em trechos:** blocos de cerca de 800 caracteres com 15% de sobreposição, cortando em fim de parágrafo ou frase. Cada trecho começa com um cabeçalho curto: título, ano e gênero.
4. **Metadados por trecho:** `movie_id`, `tmdb_id`, `ano`, `decada`, `generos`, `origem`, `diretor`, `elenco`.
5. **Embeddings:** gerar em lotes com o bge-m3 no Ollama, gravar na coluna `vector` do pgvector e criar índice HNSW.
6. **Busca textual:** gerar a coluna `tsvector` em inglês e índice GIN, para a parte por palavras-chave.

Busca (a cada pergunta)

1. **Reescrita:** o Llama transforma a fala do usuário numa busca em inglês e em filtros estruturados (ano mínimo e máximo, gênero, pessoas).
2. **Busca híbrida:** 40 trechos por similaridade vetorial e 40 por texto, já com os filtros de metadados.
3. **Fusão:** combinar as duas listas com Reciprocal Rank Fusion.
4. **Reranking:** o reranker reordena os 30 melhores trechos.
5. **Agrupar por filme:** somar os pontos dos trechos de cada filme e ficar com os 10 primeiros, excluindo os que o usuário já recusou.

Cada etapa grava no trace do Langfuse quantos itens entraram, quantos saíram e as notas, para dar para depurar por que um filme não apareceu.

Os números acima (tamanho de trecho, 40, 30, 10) são pontos de partida; a ideia é ajustá-los olhando para as métricas dos evals.

Tools do agente

O agente tem quatro tools no MVP e duas na fase 2; todas devolvem dados estruturados (Pydantic) e nenhuma altera nada fora do sistema.

Tool	Entrada	Saída	Quando o agente usa
<code>buscar_por_enredo</code>	<code>consulta_en</code> , <code>ano_min?</code> , <code>ano_max?</code> , <code>generos?</code> , <code>excluir_ids?</code> , <code>k=10</code>	Lista de candidatos: filme, ano, nota, trecho que casou	Sempre que há descrição de cena ou história
<code>buscar_filme_por_titulo</code>	<code>titulo</code> , <code>ano?</code>	Candidatos do TMDB	Quando o usuário lembra parte do nome
<code>detalhes_filme</code>	<code>tmdb_id</code>	Título em português, sinopse, pôster, diretor, 5 principais atores, gêneros	Para confirmar e montar o card da resposta
<code>buscar_pessoa</code>	<code>nome</code>	Pessoa, função e filmografia	Quando o usuário cita ator ou diretor
<code>onde_assistir</code> (fase 2)	<code>tmdb_id</code> , <code>pais="BR"</code>	Serviços de streaming, aluguel e compra	Depois de o usuário confirmar o filme
<code>filmes_similares</code> (fase 2)	<code>tmdb_id</code>	Lista de filmes parecidos	Quando o usuário pede algo parecido

Regras para as tools

- Timeout de 5 s por chamada e no máximo 1 nova tentativa.
- Erro vira mensagem estruturada para o LLM (`{"erro": "tmdb_indisponivel"}`), nunca exceção que derruba o grafo.
- Respostas do TMDB ficam 24 h em cache na memória do agent.

- No máximo 6 chamadas de tool por turno, para evitar laços.
- Cada tool tem teste unitário sem LLM, com respostas do TMDB gravadas em arquivo.

Modelo de dados

As conversas vivem só no navegador (IndexedDB); o servidor guarda apenas o índice de filmes e o feedback anônimo, sem nenhuma tabela de usuários.

No navegador (IndexedDB, banco `cinedetetive`)

Store	Campos principais	Para quê
<code>sessao</code>	<code>session_id</code> (uuid), <code>criada_em</code>	Identifica o navegador de forma anônima
<code>conversas</code>	<code>id</code> (uuid), <code>titulo</code> , <code>status</code> (aberta, resolvida), <code>filme_resolvido</code> , <code>estado</code> (json da busca), <code>criada_em</code> , <code>atualizada_em</code>	Lista lateral de conversas e estado para retomar
<code>mensagens</code>	<code>id</code> , <code>conversa_id</code> , <code>papel</code> (user, assistant), <code>conteudo</code> , <code>candidatos</code> , <code>trace_id</code> , <code>criada_em</code>	Histórico exibido no chat

Quanto tempo o histórico dura

- Na primeira visita o front chama `navigator.storage.persist()`. Se o navegador aceitar, os dados só somem se o usuário limpar os dados do site.
- Sem essa permissão, o navegador pode apagar os dados quando faltar espaço.
- O Safari costuma apagar dados de sites não visitados por cerca de 7 dias; o app deve avisar isso.
- Guia anônima: o histórico some ao fechar a janela. Outro navegador ou dispositivo: sessão nova.
- Fase 2: botão para exportar e importar conversas em JSON.

No servidor (PostgreSQL, schema `rag`)

Tabela	Colunas principais	Para quê
<code>movies</code>	<code>id</code> , <code>tmdb_id</code> , <code>match_score</code> , <code>title</code> , <code>year</code> , <code>origin</code> , <code>director</code> , <code>cast</code> , <code>genres</code> , <code>wiki_url</code>	Um registro por filme
<code>movie_chunks</code>	<code>id</code> , <code>movie_id</code> , <code>chunk_index</code> , <code>content</code> , <code>embedding</code> (vector), <code>tsv</code> (tsvector)	Trechos indexados para a busca híbrida
<code>feedback</code>	<code>id</code> , <code>session_id</code> , <code>trace_id</code> , <code>descricao</code> , <code>rating</code> (-1 ou 1), <code>correct_tmdb_id</code> , <code>comment</code> , <code>created_at</code>	Nota anônima; vira caso de teste novo

O `feedback` guarda a descrição que o usuário escreveu, para reaproveitá-la nos evals. A tela de feedback deve dizer isso de forma clara. O `trace_id` liga a nota ao trace no Langfuse, e o `session_id` agrupa os traces de um mesmo navegador.

API

O front chama direto o FastAPI do agente, com três rotas; o chat responde por streaming (Server-Sent Events).

Rotas (agent, FastAPI)

Método	Rota	O que faz
POST	<code>/v1/chat</code>	Recebe a mensagem e o contexto; responde em SSE
POST	<code>/v1/feedback</code>	Grava nota anônima e, se houver, o filme correto
GET	<code>/v1/health</code>	Verifica Postgres e Ollama

Corpo do /v1/chat

```
{
  "session_id": "uuid gerado no navegador",
  "conversa_id": "uuid da conversa",
  "mensagem": "um filme em que o cara vive o mesmo dia...",
  "historico": [{"papel": "user", "conteudo": "..."}],
  "estado": {"filtros": {}, "excluidos": [], "perguntas_feitas": 0}
}
```

Proteções

- CORS liberado só para o endereço do front.
- Limite de requisições por `session_id` e por IP (slowapi), por exemplo 20 mensagens por minuto.
- Mensagem de até 1.000 caracteres e histórico de até 10 mensagens; o resto é recusado com erro 422.
- O `estado` vindo do navegador é validado com Pydantic, pois pode ter sido alterado.

Eventos do streaming

Evento	Conteúdo	O front faz
<code>status</code>	Etapa atual ("buscando sinopses", "consultando TMDb")	Mostra o que o agente está fazendo
<code>token</code>	Pedaço do texto da resposta	Vai escrevendo a mensagem
<code>candidates</code>	Até 3 filmes com pôster, ano e confiança	Mostra cards clicáveis
<code>clarification</code>	A pergunta de esclarecimento	Destaca a pergunta
<code>final</code>	Filme escolhido, <code>estado</code> atualizado, <code>trace_id</code>	Grava a mensagem e o estado no IndexedDB e libera o feedback
<code>error</code>	Código e mensagem amigável	Mostra o erro e permite tentar de novo

Avaliação (evals)

A pergunta central dos evals é simples: o filme certo apareceu em primeiro lugar? Ela é medida em três níveis, do mais barato ao mais completo.

Dataset

Grupo	Quantidade	Como é feito
Gerados	150 casos	Filmes sorteados por década, gênero e popularidade; o LLM escreve uma descrição vaga em português a partir da sinopse, sem título nem nomes; você revisa
Escritos à mão	30 casos	Você descreve filmes de memória, como um usuário real
Multi-turno	20 casos	Descrição inicial ambígua de propósito; o filme só é achado com uma pergunta de esclarecimento
Fora do escopo	10 casos	Perguntas que não são sobre achar filme
Do feedback	cresce com o uso	Respostas com nota negativa e filme correto informado

Os casos ficam em `evals/datasets/*.jsonl` (versionados no git) e são espelhados num dataset do Langfuse.

Três níveis

1. **Recuperação:** só a busca, sem LLM gerando resposta. Rápido e barato; roda a cada pull request com 30 casos.
2. **Ponta a ponta, um turno:** o grafo inteiro com a descrição inicial.
3. **Multi-turno com usuário simulado:** um segundo LLM faz o papel do usuário, conhece o filme e responde às perguntas do agente com a verdade, sem revelar o título.

Métricas e metas iniciais

Métrica	Nível	Meta inicial
Recall@20 (filme certo entre os candidatos recuperados)	1	$\geq 90\%$
Hit@1 (filme certo em primeiro)	2 e 3	$\geq 60\%$
Hit@3	2 e 3	$\geq 80\%$
MRR	1, 2 e 3	acompanhar a evolução
Perguntas de esclarecimento por acerto	3	$\leq 1,5$

Métrica	Nível	Meta inicial
Filme inexistente ou dado inventado	2 e 3	0%
Fora do escopo tratado corretamente	2	100%
Uso correto de tools (LLM como juiz)	2 e 3	≥ 90%
Latência p95 por turno	2	< 15 s (depende do hardware)
Tokens e tempo por conversa	2 e 3	acompanhar a evolução

As metas são sugestões para começar; ajuste depois de medir a primeira versão.

Regras

- Toda mudança de prompt, modelo ou parâmetro de busca gera uma rodada de evals com o nome da versão.
- O README traz uma tabela com as métricas de cada versão e o que mudou.
- Para evitar resultado inflado, as descrições geradas não podem repetir frases da sinopse; os casos escritos à mão são o placar mais confiável.

Observabilidade

Cada mensagem do usuário gera um trace no Langfuse, agrupado por conversa, mostrando cada nó do grafo, cada tool, cada chamada ao LLM, tokens e tempo.

Onde roda

Comece no Langfuse Cloud, que tem plano gratuito, para não carregar infraestrutura no início. Hospedar por conta própria é possível via Docker, mas a versão atual pede vários serviços extras (como ClickHouse e armazenamento de objetos); vale deixar para a fase 2.

O que rastrear

Item	Onde aparece
Nó do LangGraph executado e duração	Span por nó, via callback do Langfuse no LangChain
Consulta reescrita e filtros extraídos	Entrada e saída do nó <code>entender_pedido</code>
Candidatos de cada etapa da busca e suas notas	Metadados do span de recuperação

Item	Onde aparece
Chamadas de tool, argumentos e erros	Span por tool
Confiança e caminho escolhido	Metadados do nó <code>avaliar_confianca</code>
Navegador (<code>session_id</code> anônimo), conversa, versão do app, do prompt e do modelo	<code>user_id</code> , <code>session_id</code> , <code>tags</code>
Nota do usuário	Score enviado ao trace quando chega o feedback
Resultado dos evals	Experimentos sobre o dataset, um por versão

Prompts

Os prompts ficam no gerenciamento de prompts do Langfuse, com versão. O trace registra qual versão respondeu, e os evals comparam versões lado a lado.

Painéis e alertas

- Painel diário: conversas, tokens, latência p95, taxa de nota negativa.
- Alerta (fase 2): latência p95 acima de 20 s ou taxa de erro de tool acima de 5%.
- Logs do agent com o mesmo `X-Request-Id`, gerado por requisição, para cruzar um erro com o trace.

Estrutura de pastas

Um monorepo com duas aplicações dentro de `apps/`.

```

cinedetetive/
├── apps/
│   ├── web/                # Next.js (TypeScript)
│   │   ├── app/            # rotas: / e /c/[conversaId]
│   │   ├── components/     # ChatWindow, MessageBubble, MovieCard,
FeedbackButtons, ConversationList
│   │   └── lib/
│   │       ├── db.ts       # IndexedDB com Dexie (sessao, conversas,
mensagens)
│   │       ├── session.ts  # gera o session_id e pede
storage.persist()
│   │       └── agent-client.ts # chamada ao /v1/chat e leitor de SSE
│   └── agent/              # Python
│       ├── src/cinedetetive/
│       │   ├── api/        # FastAPI: main.py, sse.py, schemas.py,

```

```

rate_limit.py
|   |   |   | graph/           # state.py, graph.py, nodes/
|   |   |   | tools/          # enredo.py, tmdb.py
|   |   |   | rag/            # retriever.py, fusion.py, reranker.py
|   |   |   | llm/            # ollama.py, prompts.py
|   |   |   | feedback.py     # grava a tabela feedback
|   |   |   | observability.py # configuração do Langfuse
|   |   |   | config.py       # variáveis de ambiente (pydantic-settings)
|   |   | ingestion/         # download.py, clean.py, link_tmdb.py,
chunk_embed.py
|   |   | evals/
|   |   | | datasets/         # casos em .jsonl
|   |   | | generate_cases.py
|   |   | | simulated_user.py
|   |   | | run_retrieval.py
|   |   | | run_e2e.py
|   |   | tests/              # testes unitários (tools, rag, nós)
|   |   | pyproject.toml      # dependências (uv)
|   | infra/
|   | | docker-compose.yml
|   | | postgres/init.sql     # cria o schema rag e a extensão vector
|   | docs/
|   | | adr/                  # registro de decisões de arquitetura
|   | .github/workflows/      # ci-web.yml, ci-agent.yml, evals.yml
|   | Makefile                # atalhos: up, models, ingest, eval, test
|   | README.md

```

Ambiente local

Tudo sobe com `make up`; só o TMDB e o Langfuse (Cloud) ficam fora da máquina.

Serviços do Docker Compose

Serviço	Imagem / build	Porta local
postgres	pgvector/pgvector:pg16	5432
ollama	ollama/ollama	11434
agent	build de apps/agent	8001
web	build de apps/web	3000

Hardware e Ollama

- Um Llama de 8B quantizado precisa de algo em torno de 8 GB de memória livre (aproximado); com GPU a resposta é bem mais rápida.
- Linux com GPU NVIDIA: o serviço `ollama` do Compose usa a GPU com o NVIDIA Container Toolkit.
- macOS: o Docker não acessa a GPU do Mac. Rode o Ollama instalado direto no sistema e aponte `OLLAMA_BASE_URL` para ele.
- Máquina mais fraca: troque para um Llama menor (por exemplo, 3B) e compare nos evals quanto o acerto cai.

Variáveis de ambiente

Variável	Usada por	Para quê
<code>DATABASE_URL</code>	agent	Conexão com o Postgres
<code>OLLAMA_BASE_URL</code>	agent, ingestion	Endereço do Ollama
<code>LLM_MODEL</code> , <code>EMBEDDING_MODEL</code>	agent, ingestion	Nomes dos modelos no Ollama
<code>TMDB_READ_TOKEN</code>	agent, ingestion	API do TMDB
<code>LANGFUSE_PUBLIC_KEY</code> , <code>LANGFUSE_SECRET_KEY</code> , <code>LANGFUSE_HOST</code>	agent, evals	Observabilidade
<code>CORS_ORIGIN</code>	agent	Endereço permitido do front
<code>NEXT_PUBLIC_AGENT_URL</code>	web	Endereço do agente

Um `.env.example` na raiz lista todas; o `.env` real fica fora do git.

Comandos do Makefile

Comando	O que faz
<code>make up</code> / <code>make down</code>	Sobe e derruba os serviços
<code>make models</code>	Baixa o Llama e o bge-m3 no Ollama (<code>ollama pull</code>)

Comando	O que faz
<code>make ingest</code>	Baixa o dataset, liga ao TMDB e gera os embeddings
<code>make test</code>	Testes de web e agent
<code>make eval-fast</code>	Evals de recuperação com 30 casos
<code>make eval-full</code>	Todos os níveis e casos, publicados no Langfuse

Roadmap

O MVP leva 6 semanas e começa pelos dados e pela busca, para que os evals existam antes do agente.

MVP em 6 semanas, com evals desde a segunda

Uma fase por semana; a fase 2 começa depois do MVP

Semana 1 · Dados
Docker, Postgres e Ollama
Ingestão: limpar, ligar ao TMDB, trechos e embeddings

Semana 2 · Busca
Busca híbrida, fusão e rerank
Casos de teste e eval no CI
Meta: Recall@20 ≥ 90%

Semana 3 · Agente
Grafo no LangGraph e 4 tools
Pergunta de esclarecimento
Langfuse ligado desde já

Semana 4 · Front
Next.js: chat com streaming
Histórico no IndexedDB
Cards de filme e feedback

Semana 5 · Modelos
Comparar Llama 8B e menor
Limites e proteções da API
Aviso sobre duração do histórico

Semana 6 · Evals
Usuário simulado (nível 3)
Ajustes guiados por métricas
Meta: Hit@1 ≥ 60%

Fase 2 · depois do MVP
Onde assistir · filmes parecidos · filmes recentes do TMDB · deploy em nuvem · Langfuse próprio

roadmap · 6 semanas de MVP e a fase 2

A ordem é proposital: medir a busca na semana 2 mostra se o RAG funciona antes de investir no grafo e no front. Se uma meta não for atingida, vale gastar mais uma semana naquela fase antes de seguir.

Decisões técnicas, riscos e próximos passos

Decisões

Decisão	Escolha	Alternativa descartada	Motivo
Identidade do usuário	Sem login, <code>session_id</code> aleatório no navegador	Cadastro com e-mail e senha	Menos código e nenhum dado pessoal guardado no servidor
Histórico	IndexedDB no navegador	Tabelas de conversa no Postgres	Some com a guia anônima ou outro navegador, como desejado; servidor fica sem estado
Backend	Só o FastAPI do agente	Laravel na frente do agente	Sem contas nem histórico, a API de produto não teria função
Modelos	Llama, bge-m3 e reranker locais (Ollama)	Gemini via API	Sem custo por token e sem enviar textos a um provedor de LLM
Estado da conversa	Enviado pelo navegador a cada mensagem	Checkpointing do LangGraph no servidor	Coerente com o histórico local; o servidor reinicia sem perder nada
Banco vetorial	pgvector	ChromaDB	Busca híbrida numa consulta só; o mesmo banco guarda o feedback
Idioma da busca	Reescrever a consulta em inglês	Traduzir as 35 mil sinopses	Mais barato; embeddings multilíngues cobrem o resto

Cada decisão vira um arquivo curto em `docs/adr/`.

Riscos

Risco	Efeito	Mitigação
Llama local menos preciso em tool calling que modelos grandes	Tools com argumentos errados, mais erros	Saída estruturada com validação e nova tentativa; medir "uso correto de tools" nos evals
Hardware fraco	Respostas lentas, evals demorados	Modelo menor, reranker só nos 20 primeiros, nível 3 dos evals rodado à noite
Navegador apaga o histórico	Usuário perde conversas	Pedir <code>storage.persist()</code> , avisar sobre o Safari, exportar JSON na fase 2
Estado adulterado pelo navegador	Requisições inválidas	Validar com Pydantic e impor limites de tamanho
Dataset cobre pouco os filmes recentes	Agente não acha lançamentos novos	Fase 2: indexar sinopses curtas do TMDb dos filmes recentes mais populares
Pareamento errado Wikipedia → TMDb	Pôster ou elenco de outro filme	Guardar <code>match_score</code> , exigir título e ano compatíveis
Evals inflados por descrições geradas	Métrica boa e uso real ruim	Proibir frases copiadas da sinopse; usar os casos escritos à mão como placar principal
Textos nos traces do Langfuse Cloud	Conversas saem da máquina	Langfuse próprio ou traces sem entradas e saídas

Próximos passos

- ☐ Instalar o Ollama e baixar o Llama e o bge-m3
- ☐ Criar o repositório e o `docker-compose.yml` com Postgres
- ☐ Pedir a chave do TMDb e baixar o dataset do Kaggle
- ☐ Escrever os 30 casos de teste à mão antes de codar a busca
- ☐ Implementar a ingestão e rodar o primeiro eval de recuperação